

PHƯƠNG PHÁP PHÁT TRIỂN PHẦN MỀM HƯỚNG ĐỐI TƯỢNG VỚI UML

Bài 1: Giới thiệu quy trình ICONIX Process

Nội dung chính:

Bài này cung cấp cái nhìn tổng quan về **Quy trình ICONIX**, một phương pháp tiếp cận phân tích và thiết kế hướng đối tượng theo hướng tối giản, tập trung vào việc **kết nối chặt chẽ giữa yêu cầu (use case) và mã nguồn** trong phát triển phần mềm. Quy trình ICONIX đặc biệt hữu ích trong các dự án phần mềm hướng đối tượng khi cần chuyển đổi có hệ thống từ yêu cầu của khách hàng thành mô hình thiết kế rõ ràng và mã nguồn chất lượng. Bài này cũng giới thiệu bối cảnh ra đời của ICONIX như một giải pháp thực dụng giữa hai cực: UML đầy đủ nhưng cồng kềnh và các phương pháp Agile thường bỏ qua thiết kế chi tiết. Bên cạnh đó, bài học trình bày cấu trúc tổng thể của ICONIX (4 giai đoạn – 4 mốc kiểm duyệt), cùng với vai trò nền tảng của giai đoạn yêu cầu và mô hình hóa (domain model, use case, GUI storyboard) trong toàn bộ quy trình phát triển.

Mục tiêu cần đạt

- Hiểu bối cảnh ra đời, bản chất và vai trò của Quy trình ICONIX trong phát triển phần mềm hướng đối tượng.
- Nắm được 4 giai đoạn chính của ICONIX Process, từ xác định yêu cầu đến triển khai.
- Hiểu được ý nghĩa của 4 mốc kiểm duyệt và vai trò của chúng trong kiểm soát chất lượng phần mềm.
- Nắm được cách thức chuyển đổi từ use case sang thiết kế và mã nguồn, bao gồm các bước trung gian như domain modeling, robustness analysis và sequence diagram.
- Nhận biết được tầm quan trọng của giai đoạn yêu cầu và mô hình hóa như nền tảng cho toàn bộ thiết kế và lập trình sau này.

1.1. Vấn đề và sự ra đời của ICONIX

Trong suốt lịch sử phát triển phần mềm, các bạn sẽ thấy có một nghịch lý quen thuộc: những quy trình được đề xuất để quản lý sự phức tạp của phần mềm thường hoặc quá lý tưởng hóa, hoặc quá cồng kềnh để áp dụng trong thực tế. UML (Unified Modeling Language) — ngôn ngữ mô hình hóa thống nhất — là một ví dụ điển hình. Khi mới xuất hiện vào thập niên 1990, UML được kỳ vọng sẽ là “*ngôn ngữ chung*” để các nhóm phát triển phần mềm trên toàn thế giới giao tiếp với nhau. Nó cung cấp một bộ công cụ biểu diễn cực kỳ phong phú: từ biểu đồ lớp,

biểu đồ use case, biểu đồ tuần tự, trạng thái, hoạt động... cho đến những khái niệm phức tạp như các gói (package), thành phần (component) và triển khai (deployment).

354 / 391



100%



come to master it. Depending on your background, there are different ways to approach using the UML for the first time. As you gain more experience, you will come to understand and appreciate its more subtle parts.

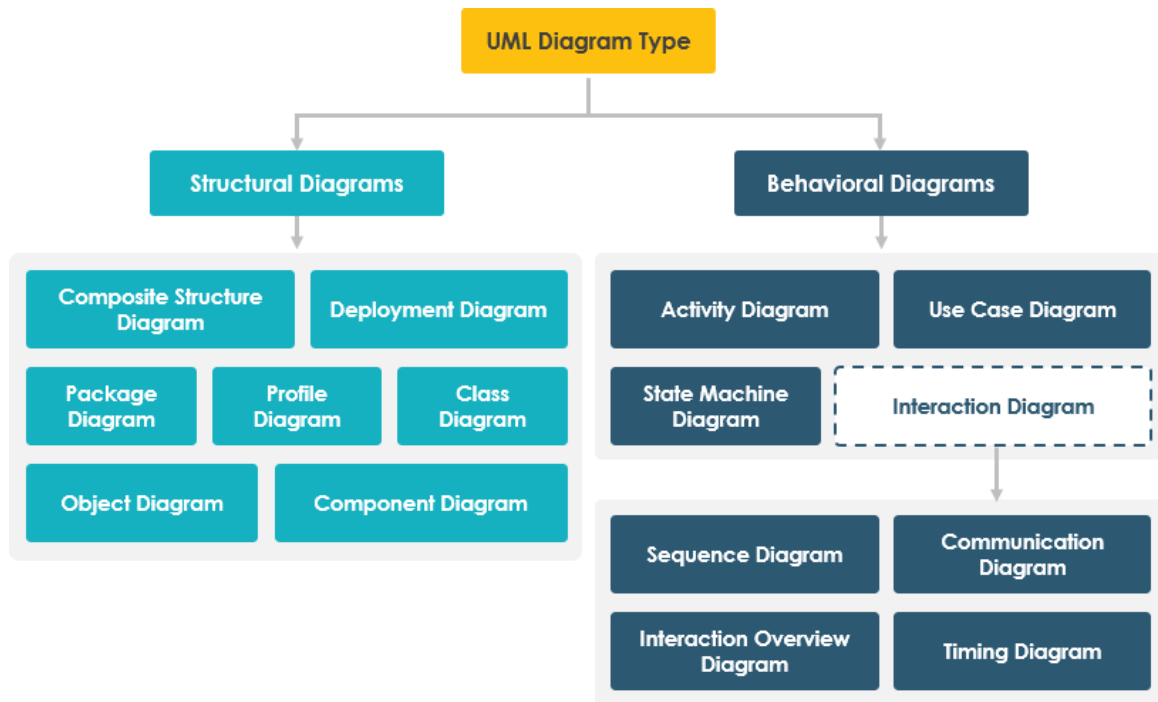
If you can think it, the UML can model it.

Transitioning to the UML

You can model 80 percent of most problems by using about 20 percent of the UML. Basic structural things, such as classes, attributes, operations, use cases, components, and packages, together with basic structural relationships, such as dependency, generalization, and association, are sufficient to create static models for many kinds of problem domains. Add to that list basic behavioral things, such as simple state machines and interactions, and you can model many useful aspects of a system's dynamics. You'll need to use only the more advanced features of the UML once you start modeling the things you encounter in more-complex situations, such as modeling concurrency and distribution.

Trong cuốn sách *The Unified Modeling Language User Guide* của Grady Booch, James Rumbaugh và Ivar Jacobson (Addison-Wesley, 1998), có một nhận định rằng “**có thể thực hiện 80% việc mô hình hóa chỉ với 20% UML**”, nằm ở một phần sau trang 400. Nếu nhận định này được nêu ra ngay từ phần giới thiệu, nó có thể đã giúp ngành công nghiệp tiết kiệm hàng triệu, thậm chí hàng tỷ đô la, và tránh được những trường hợp "phân tích tê liệt". Tuy nhiên, điều đáng tiếc là **họ không nói rõ 20% UML nào là phần hữu ích nhất**.

Tuy nhiên, cũng chính sự phong phú đó đã tạo nên một rào cản lớn. Trong nhiều dự án thực tế, người ta phát hiện ra rằng nếu cố gắng áp dụng “đầy đủ” tất cả các bước và biểu đồ UML theo sách vở, thì nhóm sẽ mất quá nhiều thời gian vào việc mô hình hóa mà không còn thời gian để... viết phần mềm thật sự. Quá trình mô hình hóa trở nên nặng nề, xa rời thực tiễn, và thường bị quản lý dự án cắt giảm đầu tiên khi tiến độ bị chậm. Đếm sơ sơ ta cũng đã có 14 loại sơ đồ như hình bên dưới.



1.1.1 Vấn đề “nhảy vào code” quá sớm

Các bạn thử tưởng tượng một nhóm phát triển ở một công ty phần mềm nhỏ. Họ nhận một dự án xây dựng hệ thống đặt vé xe khách cho một hãng vận tải địa phương. Sau vài buổi họp ngắn với khách hàng, nhóm lập tức lao vào chia việc viết code: người thì thiết kế database, người thì code phần quản trị, người thì dựng giao diện web... Nghe có vẻ năng động, nhưng thực chất họ đã bỏ qua một giai đoạn quan trọng — phân tích và thiết kế có hệ thống.

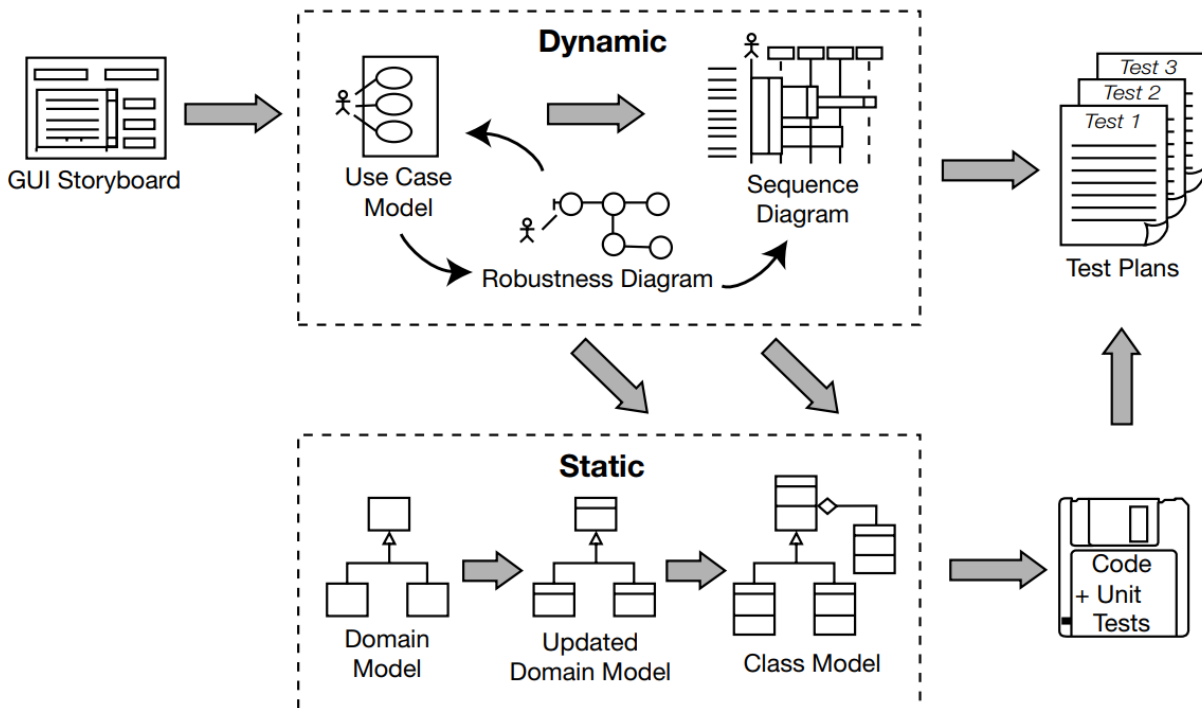
Hệ quả không khó đoán:

- Sau vài tuần, những mâu thuẫn trong cách hiểu yêu cầu bắt đầu lộ ra. Bộ phận giao diện hiểu “đặt vé” là một quá trình đơn giản, nhưng bộ phận backend lại nhận ra có rất nhiều trường hợp ngoại lệ (ví dụ vé bị giữ nhưng chưa thanh toán, hoặc khách đổi chuyến).
- Database đã thiết kế xong nhưng không phản ánh đúng các thực thể quan trọng trong nghiệp vụ (như vé, chuyến xe, khách hàng, lịch sử giao dịch), dẫn đến phải chỉnh sửa lớn sau này.
- Các lập trình viên viết code dựa trên những giả định không nhất quán, khiến hệ thống nhanh chóng trở nên rối rắm.

Tình trạng này không phải là cá biệt. Trong giới phát triển, nó được gọi bằng một cụm từ khá chua chát: “jumping to code” — nhảy vào code quá sớm, trước khi thực sự hiểu bài toán. Nguyên nhân rất thực tế: tiến độ dự án thường được đánh giá bằng số dòng code đã sinh ra, chứ không phải bằng mức độ rõ ràng của yêu cầu hay chất lượng thiết kế. Quản lý thường

muốn “thấy cái gì đó chạy được” để báo cáo tiến độ. Và vì vậy, nhóm phát triển bị đẩy vào một guồng máy quen thuộc: hiểu sơ sơ → viết code → gặp lỗi → vá lỗi → sửa thiết kế “trên không trung”.

ICONIX Process ra đời chính là để giải quyết cái khoảng trống chết người này.



1.1.2 Khoảng trống giữa “Use Case” và “Code”

Nếu UML toàn diện là một “quái vật đồ sộ”, thì ICONIX chọn một cách tiếp cận khác: tối giản và tập trung. Các tác giả ICONIX nhận thấy rằng phần khó nhất trong quy trình phát triển phần mềm không phải là viết code, mà là giai đoạn ở giữa — từ lúc có use case đến lúc bắt tay vào lập trình.

Hãy xem xét kỹ hơn:

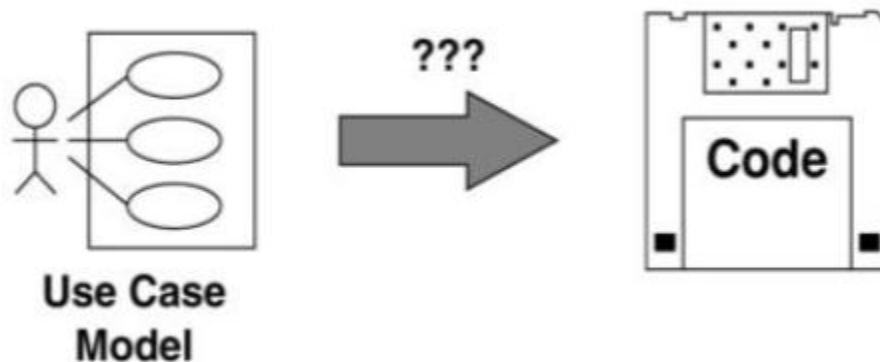
- Use case cho chúng ta biết hệ thống sẽ được sử dụng như thế nào — ai làm gì, hệ thống phản ứng ra sao.
- Code là hiện thực cụ thể của hệ thống đó.

Ở giữa hai điểm này là một vùng xám mênh mông, nơi các quyết định thiết kế quan trọng phải được đưa ra:

- Chúng ta sẽ phân rã yêu cầu thành các lớp, đối tượng như thế nào?
- Dữ liệu sẽ được mô hình hóa ra sao?

- Ai chịu trách nhiệm xử lý logic nghiệp vụ nào?
- Làm sao đảm bảo khi nhiều lập trình viên cùng làm, hệ thống không biến thành “mì spaghetti”?

UML cung cấp nhiều biểu đồ để hỗ trợ giai đoạn này, nhưng như các bạn thấy, nó quá nhiều lựa chọn — mà thời gian dự án thì luôn hạn hẹp. Trong khi đó, các quy trình Agile thường bỏ qua hoặc làm rất sơ sài phần này để “tiết kiệm thời gian”, dẫn đến hệ thống thiếu nền tảng vững chắc.



ICONIX Process tập trung đúng vào khoảng trống đó, với một chuỗi bước tối giản nhưng có trình tự rõ ràng để đưa nhóm từ use case → thiết kế → code mà không bị ngắt quãng.

1.1.3 Vì sao cần một “cookbook”

Một điều thú vị là ICONIX không phủ nhận sự phức tạp của phát triển phần mềm. Ngược lại, nó chấp nhận rằng mỗi dự án là duy nhất, mỗi nhóm có hoàn cảnh riêng. Tuy nhiên, chính vì vậy mà nhóm phát triển rất dễ lạc lối trong hàng loạt quyết định nhỏ hằng ngày. Một quy trình quá “mở” sẽ dẫn đến việc mỗi thành viên tự hiểu và tự xử lý theo cách riêng → hệ thống mất tính nhất quán.

Tác giả ICONIX kể rằng khi họ bắt đầu dạy mô hình hóa hướng đối tượng vào đầu những năm 1990, một khách hàng từng nói:

“Hãy viết một cuốn cookbook (sách nấu ăn). Nhân viên của tôi thích làm theo các bước rõ ràng, không phải phỏng đoán.”

Và đó là tinh thần của ICONIX: không áp đặt một bộ quy tắc cứng nhắc, nhưng cung cấp một trình tự các bước có thể lặp lại được (**repeatable sequence of steps**) để dẫn dắt nhóm qua những giai đoạn mơ hồ nhất.

Các bạn hãy hình dung: khi đứng trước một yêu cầu phức tạp, việc có sẵn một “công thức” sẽ giống như bạn đang nấu ăn có sẵn hướng dẫn từng bước. Bạn có thể thêm nếm gia vị theo khẩu vị, nhưng khung cơ bản thì rõ ràng, giúp bạn không bị rối.

1.1.4 ICONIX và thực tiễn doanh nghiệp

ICONIX không sinh ra trong phòng thí nghiệm hàn lâm, mà trong môi trường doanh nghiệp thật, nơi tiến độ gấp, tài nguyên hạn chế, và khách hàng thay đổi yêu cầu liên tục. Quy trình này đã được áp dụng trong hàng trăm dự án lớn nhỏ trong vòng hơn 10 năm và chứng minh được ba điều quan trọng:

- Nó đủ nhẹ để không làm chậm tiến độ. Không cần vẽ toàn bộ 13 loại biểu đồ UML; ICONIX chỉ tập trung vào vài loại thiết yếu (use case, domain model, robustness diagram, sequence diagram) theo đúng thời điểm cần thiết.
- Nó đủ có cấu trúc để tránh hỗn loạn. Các bước trong ICONIX như yêu cầu → mô hình miền → robustness → thiết kế chi tiết → kiểm thử được nối kết rõ ràng, có điểm kiểm duyệt (milestone) cụ thể. Điều này cực kỳ quan trọng khi các bạn làm việc theo nhóm hoặc phải bàn giao giữa nhiều giai đoạn.
- Nó phù hợp với Agile. ICONIX không mâu thuẫn với Agile; ngược lại, nó bổ sung vào chỗ Agile thường bỏ trống. Các iteration nhỏ có thể đi qua đủ vòng ICONIX cho một cụm use case nhỏ, giúp phản hồi nhanh mà không đánh mất chất lượng thiết kế.

Khi các bạn học về UML hay phân tích thiết kế phần mềm, có thể nhiều bạn cảm thấy: *“Đây toàn là lý thuyết, vào dự án thật thì chẳng ai làm thế này.”* Và đúng là trong nhiều doanh nghiệp Việt Nam hiện nay, việc lập tài liệu UML đầy đủ là điều xa xỉ. Nhưng điều đó không có nghĩa là không cần mô hình hóa — mà là ta cần **một phương pháp thực dụng hơn**.

ICONIX Process chính là phương pháp đó. Nó không yêu cầu các bạn phải trở thành “UML guru” để áp dụng, cũng không bắt buộc viết ra hàng chục tài liệu. Nó chỉ cần các bạn hiểu rõ trình tự tư duy: từ yêu cầu → hiểu bài toán → mô hình hóa → thiết kế → code → kiểm thử.

Vì vậy, trong buổi học hôm nay, chúng ta sẽ không đi sâu vào lập trình hay chi tiết kỹ thuật triển khai. Mục tiêu chính là giúp các bạn hiểu bối cảnh ra đời, vấn đề thực tế và tư tưởng cốt lõi của ICONIX Process. Các buổi tiếp theo sẽ lần lượt đi sâu vào từng giai đoạn cụ thể.

Chốt lại, ICONIX Process ra đời để giải quyết một vấn đề rất thực tế mà hầu hết các nhóm phát triển phần mềm đều gặp phải: **làm thế nào để bắt đầu một cách có hệ thống từ yêu cầu đến code, trong bối cảnh nguồn lực có hạn, thời gian bị ép, và yêu cầu thay đổi liên tục**. Nó không thay thế UML, cũng không phủ nhận Agile, mà chọn một con đường giữa: tối giản nhưng có trình tự rõ ràng, thực dụng và có thể lặp lại.

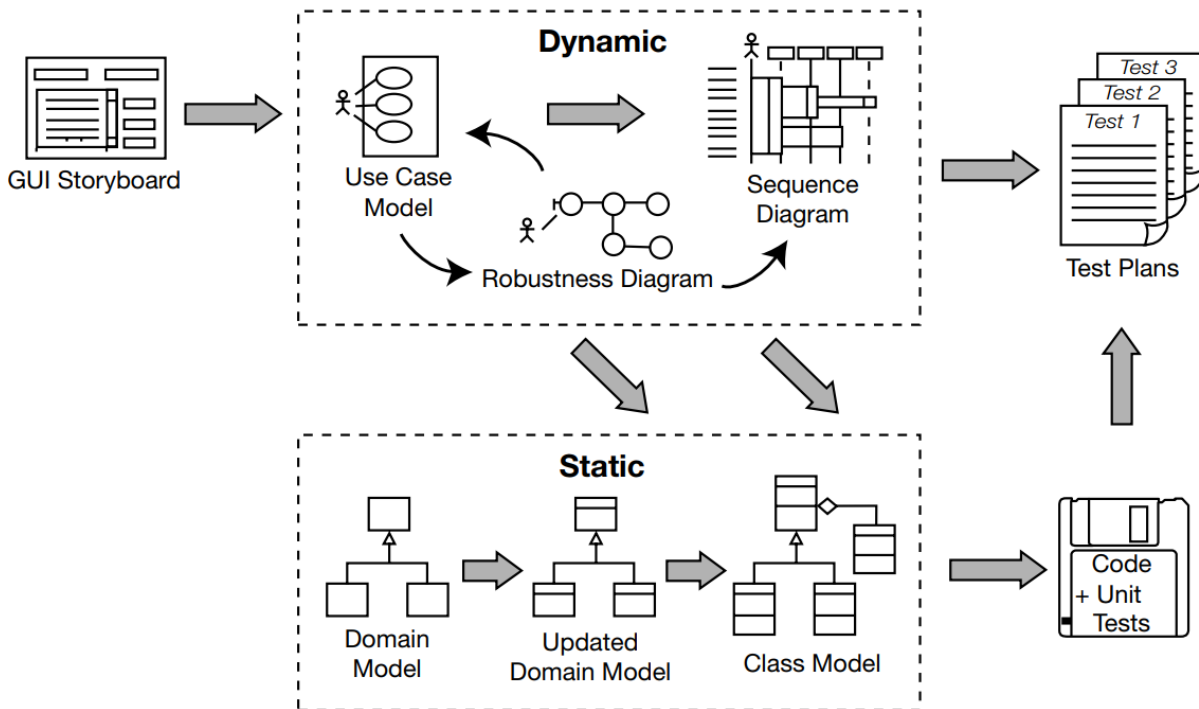
1.2. Quy trình ICONIX

Nếu như ở phần 1, các bạn đã hiểu lý do tại sao ICONIX Process ra đời — để giải quyết khoảng trống chết người giữa *use case* và *code* — thì ở phần này, chúng ta sẽ cùng nhau đi qua toàn bộ quy trình ICONIX từ đầu đến cuối như một “đường đi nước bước” rõ ràng.

Các bạn có thể tưởng tượng ICONIX giống như một bản đồ chỉ đường. Nó không quá chi tiết đến mức rườm rà như bản đồ thành phố tỉ lệ 1:1, nhưng cũng không quá sơ sài như một bản phác thảo trên khăn giấy. Nó đưa nhóm phát triển đi qua những chặng đường chính yếu, đảm bảo không bị lạc, nhưng vẫn đủ linh hoạt để thích ứng với thực tế của từng dự án.

1.2.1 Góc nhìn vĩ mô – bird eye view

Hãy hình dung một đường đi bắt đầu từ **yêu cầu người dùng (use case)** và kết thúc ở **mã nguồn đã kiểm thử (tested code)**.



Ở điểm xuất phát, chúng ta có các mô tả yêu cầu, thường là dưới dạng:

- tài liệu đặc tả chức năng (functional specification),
- hoặc các đoạn hội thoại, email, bảng Excel, sơ đồ phác thảo mà khách hàng đưa ra,
- đôi khi chỉ là vài ý tưởng rời rạc và một cái mockup sơ sài.

- ...tóm lại là một mớ bong bong chưa hoặc không có tổ chức, tùy theo cách các bạn giao tiếp với khách hàng (ví dụ: trong cuộc họp, café, hoặc trên bàn nhậu...bạn lưu thông tin khách hàng muốn trong đầu, ghi âm, tin nhắn....)

Ở điểm đích, ta có hệ thống chạy được, đã qua unit test, sẵn sàng triển khai. ICONIX Process định nghĩa rõ ràng những trạm dừng trung gian trên con đường này, bao gồm:

1. **Xác định yêu cầu (Requirements)** – hiểu chính xác hệ thống phải làm gì và người dùng tương tác ra sao.
2. **Phân tích & Thiết kế sơ bộ (Analysis / Preliminary Design)** – chuyển từ use case thành mô hình miền và sơ đồ bền vững (robustness).
3. **Thiết kế chi tiết (Detailed Design)** – vẽ sequence diagram để phân bổ hành vi vào các lớp, hoàn thiện mô hình tĩnh.
4. **Triển khai (Implementation)** – viết code và kiểm thử dựa trên những mô hình đã xây dựng.

Đây là bốn giai đoạn lớn, và giữa chúng là những “cột mốc kiểm duyệt” (milestones) — những thời điểm nhóm tạm dừng để kiểm tra xem kết quả có khớp với mong đợi không. Giống như các bạn học môn học này chẳng hạn, thi thoảng phải dừng lại để làm bài kiểm tra thường xuyên xem kiến thức của các bạn có đi đúng hướng để học những phần tiếp theo hay không

Điều đặc biệt là ICONIX không yêu cầu các bạn đi theo quy trình thác nước tuần tự. Ngược lại, nó được thiết kế để lặp lại nhiều lần cho từng cụm use case nhỏ, rất phù hợp với cách làm Agile. Mỗi vòng lặp ICONIX có thể nhỏ như vài tuần, xử lý một nhóm tính năng cụ thể, nhưng vẫn đảm bảo có kiểm soát.

1.2.2. Hai luồng công việc: Động (dynamic) và Tĩnh (static)

Để hiểu ICONIX, trước hết các bạn cần nắm rõ hai luồng công việc song song nhưng đan xen:

- **Luồng động (Dynamic):** tập trung vào hành vi của hệ thống, gồm use case, GUI storyboard, robustness diagram, sequence diagram. Đây là luồng trả lời câu hỏi: *“Hệ thống phản ứng thế nào khi người dùng làm điều này?”*
- **Luồng tĩnh (Static):** tập trung vào cấu trúc của hệ thống, gồm domain model và class diagram. Đây là luồng trả lời câu hỏi: *“Hệ thống gồm những thành phần gì, chúng liên kết với nhau như thế nào?”*

Hai luồng này không chạy tuần tự mà liên tục tác động qua lại. Ví dụ: khi các bạn vẽ robustness diagram (thuộc luồng động), các bạn sẽ phát hiện ra có những lớp miền (domain

class) còn thiếu → phải quay lại cập nhật domain model (thuộc luồng tĩnh). Ngược lại, khi thiết kế class diagram, các bạn sẽ thấy có những hành vi chưa được mô tả rõ trong use case → phải quay lại bổ sung mô tả hành vi. Đây chính là điểm mạnh của ICONIX: nó chấp nhận và hỗ trợ tính lặp, thay vì giả vờ như quy trình là một đường thẳng hoàn hảo.

1.2.3. Bốn giai đoạn chính

(1) Giai đoạn Xác định Yêu cầu

Đây là nơi nhóm phát triển và khách hàng cùng nhau làm rõ “chúng ta sẽ xây cái gì”. Giai đoạn này bao gồm ba hoạt động chính:

- Xác định yêu cầu chức năng (functional requirements): hệ thống cần làm gì, những chức năng bắt buộc là gì.
- Xây dựng mô hình miền (domain model) bước đầu: nhận diện các khái niệm, thực thể, mối quan hệ quan trọng trong bài toán.
- Viết use case bản nháp và dựng GUI storyboard: mô tả cách người dùng tương tác với hệ thống.

Tất cả các hoạt động này đều dẫn tới Cột mốc 1 – Requirements Review, nơi nhóm kiểm tra xem đã hiểu đúng yêu cầu của khách hàng chưa. Đây chính là giai đoạn “cắm mốc” cho cả dự án. Nếu sai ở đây, mọi thứ phía sau đều sai theo.

(2) Giai đoạn Phân tích & Thiết kế sơ bộ

Một khi các use case đã rõ ràng, nhóm sẽ tiến hành Robustness Analysis — một bước đặc trưng của ICONIX.

Tại đây, các bạn sẽ biểu diễn từng use case dưới dạng sơ đồ bền vững (robustness) → từ giờ về sau sẽ gọi là sơ đồ robustness vì dịch ra tiếng Việt nghe nó ngang phè phè :

- nhận diện các đối tượng biên (boundary),
- các đối tượng thực thể (entity),
- và các controller logic cần có để xử lý kịch bản.

Trong quá trình này, domain model được cập nhật và tinh chỉnh. Đây là lúc những khoảng trống trong hiểu biết về hệ thống dần được lấp đầy. Giai đoạn này kết thúc bằng Cột mốc 2 – Preliminary Design Review (PDR), nơi nhóm rà soát xem use case, robustness diagram và domain model đã khớp nhau chưa.

(3) Giai đoạn Thiết kế Chi tiết

Khi PDR đã ổn, nhóm chuyển sang vẽ sequence diagram cho từng use case. Sequence diagram chính là nơi:

- Các hành vi cụ thể được gán cho từng lớp, làm rõ vai trò và trách nhiệm của từng thành phần trong hệ thống.
- Trình tự trao đổi thông điệp giữa các đối tượng được xác định, nhằm mô tả rõ cách các đối tượng phối hợp với nhau để thực hiện một chức năng hay ca sử dụng.
- Các phương thức (operations) tương ứng được định nghĩa cho từng lớp, đảm bảo lớp có đủ khả năng thực hiện các hành vi đã được gán

Sau đó, nhóm cập nhật class diagram để phản ánh những gì đã được xác định trong sequence diagram. Giai đoạn này kết thúc bằng Cột mốc 3 – Critical Design Review (CDR), một cuộc rà soát quan trọng để đảm bảo thiết kế chi tiết thực sự khớp với yêu cầu.

(4) Giai đoạn Triển khai

Cuối cùng là viết code, viết unit test, thực hiện kiểm thử tích hợp và rà soát code để đồng bộ lại với mô hình. Đây chính là lúc thiết kế trở thành hiện thực.

Tuy nhiên, trong phạm vi khóa học này của các bạn, chúng ta sẽ không đi quá sâu vào phần viết cụ thể code và triển khai. Phần đó sẽ được học riêng trong các môn học khác. Điều tôi muốn nhấn mạnh ở đây là khi có bản thiết kế tốt việc viết code cực kỳ nhàn (sử dụng tool hoặc AI để sinh code tự động từ bản thiết kế).

1.2.4. Bốn cột mốc kiểm duyệt (Milestones)

ICONIX Process rất coi trọng kiểm duyệt ở những điểm giao quan trọng. Đây không phải là những cuộc họp hình thức, mà là các “trạm kiểm soát” thực sự:

1. Requirements Review → Đảm bảo nhóm hiểu đúng yêu cầu, use case khớp với mong đợi của khách hàng.
2. Preliminary Design Review (PDR) → Đảm bảo domain model, use case và robustness diagram nhất quán.
3. Critical Design Review (CDR) → Đảm bảo thiết kế chi tiết (sequence diagram, class diagram) đáp ứng yêu cầu, không còn “lỗ hổng logic”.
4. Code Review & Model Update → Đảm bảo code được viết khớp với mô hình, và mô hình được cập nhật tương ứng.

Bốn mốc này tạo thành khung kiểm soát chất lượng xuyên suốt quy trình, giúp nhóm không bị trượt dần mà không nhận ra. Các bạn hình dung chúng ta học nhưng cũng có kiểm tra liên tục, cả ở nhà, lẫn trên lớp, thường xuyên...

Nhiều bạn có thể thắc mắc: “Nghe quy trình này có vẻ khá bài bản, liệu có mâu thuẫn với Agile không?” Câu trả lời là không những không mâu thuẫn, mà còn bổ sung cho nhau rất tốt.

Agile thiên về chia nhỏ phạm vi, phản hồi nhanh và thích ứng với thay đổi. ICONIX cung cấp cho mỗi vòng lặp nhỏ đó một **bộ khung thiết kế có cấu trúc**, giúp đảm bảo mỗi nhóm tính năng phát triển ra đều có chất lượng, không chỉ là “chạy được”.

Ví dụ, trong một sprint 2 tuần, nhóm có thể:

- chọn 2–3 use case,
- đi trọn vẹn các bước ICONIX cho chúng,
- đến cuối sprint có code đã kiểm thử.

Lần sprint sau lại lặp lại cho nhóm use case tiếp theo. Nhờ đó, hệ thống dần được xây dựng lên một cách có tổ chức, tránh được tình trạng “vá vú” theo thời gian. Để dễ hình dung, các bạn có thể tưởng tượng ICONIX Process như một vòng lặp xoáy tròn ốc nhỏ. Mỗi vòng bắt đầu từ một nhóm use case → phân tích → thiết kế → triển khai → kiểm duyệt → rồi trở lại với nhóm use case tiếp theo. Sau vài vòng lặp, hệ thống lớn dần lên, nhưng không bị mất kiểm soát, vì mỗi vòng đều có cấu trúc và kiểm duyệt rõ ràng.

Phần này các bạn cần nắm rõ ba điểm mấu chốt: (1) ICONIX Process là một chuỗi bước có trình tự rõ ràng từ yêu cầu đến code, không thừa cũng không thiếu. (2) Nó dựa trên hai luồng công việc động và tĩnh, đan xen lặp lại, giúp giữ cho hệ thống vừa phản ánh hành vi người dùng vừa có cấu trúc vững chắc. (3) Nó phù hợp với Agile và các môi trường phát triển hiện đại, vì có thể chia nhỏ theo use case và lặp lại nhiều vòng.

1.3. Giai đoạn yêu cầu & mô hình hóa

Khi bắt đầu một dự án phần mềm, điều quan trọng nhất không phải là viết code nhanh, mà là hiểu đúng và đủ vấn đề cần giải quyết. ICONIX Process đặc biệt nhấn mạnh giai đoạn này, bởi vì nó là nền móng của toàn bộ thiết kế phía sau. Nếu giai đoạn yêu cầu làm hời hợt, thì mọi bước tiếp theo — dù có kỹ thuật tốt đến mấy — cũng sẽ đứng trên một nền đất lung lay.

1.3.1. Functional vs. Behavioral Requirements — hiểu đúng “hệ thống phải làm gì”

Hầu hết các dự án phần mềm đều bắt đầu bằng một danh sách yêu cầu chức năng (functional requirements). Đây là danh sách mà khách hàng hoặc nhóm business thường đưa ra, với nội dung kiểu như:

- “Hệ thống phải cho phép người dùng đặt vé.”

- “Hệ thống phải lưu trữ thông tin khách hàng.”
- “Hệ thống phải gửi email xác nhận.”

Thoạt nhìn, danh sách này có vẻ rõ ràng. Nhưng trên thực tế, nếu các bạn đưa nó cho nhóm lập trình và nói “hãy làm theo đây”, thì hệ quả thường là hỗn loạn. Lý do là vì functional requirements chỉ nói “cái gì”, chứ không nói “như thế nào”.

Ví dụ: câu “Hệ thống phải cho phép người dùng đặt vé” tưởng đơn giản, nhưng có vô số câu hỏi chưa được trả lời:

- Người dùng là ai? Có những loại người dùng nào?
- Quy trình đặt vé gồm những bước nào? Có thể hủy không? Có giữ chỗ tạm thời không?
- Nếu hai người đặt cùng một ghế thì sao?
- Có khác nhau giữa đặt vé trực tuyến và tại quầy không?

Functional requirements giống như một bản danh sách “món ăn” mà khách hàng muốn, nhưng không cho ta biết cách nấu, nguyên liệu, hay quy trình thực hiện.

Chính vì vậy, ICONIX Process đưa thêm một khái niệm quan trọng: behavioral requirements — các yêu cầu hành vi. Đây là mô tả chi tiết cách người dùng tương tác với hệ thống và hệ thống phản hồi ra sao, thường được thể hiện bằng use case và storyboard GUI.

Nếu functional requirements trả lời câu hỏi “hệ thống phải có những chức năng gì?”, thì behavioral requirements trả lời câu hỏi “người dùng và hệ thống sẽ tương tác như thế nào trong thực tế?”.

1.3.2. Domain Modeling — xây dựng ngôn ngữ chung

Trước khi viết use case, ICONIX khuyến khích một bước rất quan trọng: domain modeling — mô hình hóa miền bài toán.

Hãy thử hình dung: trong một nhóm phát triển có nhiều thành viên, mỗi người đến từ một chuyên môn khác nhau (lập trình, thiết kế UI, kiểm thử, phân tích nghiệp vụ...). Nếu không có một ngôn ngữ chung để nói về các khái niệm trong hệ thống, mỗi người sẽ hiểu theo cách riêng → dẫn tới sai lệch. Với thể hệ của các bạn thì sao???, chắc chỉ có các bạn mới hiểu sigma, fishu, slay... Khi tất cả các thể hệ ngồi vào mâm cơm, cần một ngôn ngữ để nói ai cũng hiểu, phần mềm cũng vậy. Domain model chính là “từ điển chung” cho cả dự án.

Nó là một sơ đồ đơn giản (thường là biểu đồ lớp không có chi tiết kỹ thuật) thể hiện:

- Những khái niệm, thực thể quan trọng trong hệ thống.

- Quan hệ giữa chúng (quan hệ “là một” – is-a, quan hệ “có” – has-a).

Ví dụ, với hệ thống đặt vé xe khách, domain model có thể gồm các lớp:

- Khách hàng (Customer)
- Chuyến xe (Trip)
- Vé (Ticket)
- Xe (Bus)
- Tuyến đường (Route)

Quan hệ có thể như sau:

- Một *Chuyến xe* có nhiều *Vé*.
- Một *Khách hàng* đặt nhiều *Vé*.
- Một *Tuyến đường* bao gồm nhiều *Chuyến xe*.

Domain model không phải database diagram. Nó không chứa chi tiết kỹ thuật, index, kiểu dữ liệu... mà tập trung vào ngôn ngữ miền nghiệp vụ. Đây là điểm nhiều sinh viên hay nhầm: domain model không phải là ERD (entity-relationship diagram). Nó mang tính khái niệm, không phải vật lý.

Tác giả ICONIX nhấn mạnh rằng phiên bản đầu tiên của domain model chắc chắn sẽ sai — và điều đó hoàn toàn bình thường. Khi các bạn phân tích use case sâu hơn, domain model sẽ được tinh chỉnh dần.

1.3.3. Use Case Modeling — kể lại câu chuyện tương tác

Khi domain model đã có, nhóm sẽ viết use case — mô tả các kịch bản sử dụng hệ thống từ góc nhìn người dùng. Một use case không phải là tài liệu kỹ thuật khô khan, mà là một câu chuyện có trình tự:

- Ai là người tham gia? (Actor)
- Họ muốn đạt mục tiêu gì?
- Họ thực hiện các bước gì?
- Hệ thống phản hồi ra sao?
- Có những trường hợp ngoại lệ nào?

Ví dụ, Use Case “Đặt vé xe” có thể được viết như sau:

Tác nhân chính: Khách hàng

Mục tiêu: Đặt vé xe thành công cho chuyến đi mong muốn.

Luồng cơ bản:

Khách hàng chọn tuyến đường và ngày đi. Hệ thống hiển thị danh sách chuyến xe phù hợp.

Khách hàng chọn chuyến xe và chỗ ngồi. Hệ thống giữ vé tạm thời và yêu cầu thanh toán.

Khách hàng thanh toán thành công. Hệ thống xác nhận và gửi email vé điện tử.

Luồng ngoại lệ:

- Nếu thanh toán thất bại, hệ thống hủy giữ vé.
- Nếu ghế vừa được người khác đặt trước đó, hệ thống yêu cầu khách chọn lại.

Nhờ cách viết như vậy, mọi người trong nhóm — kể cả không kỹ thuật — đều có thể hiểu rõ hành vi mong đợi.

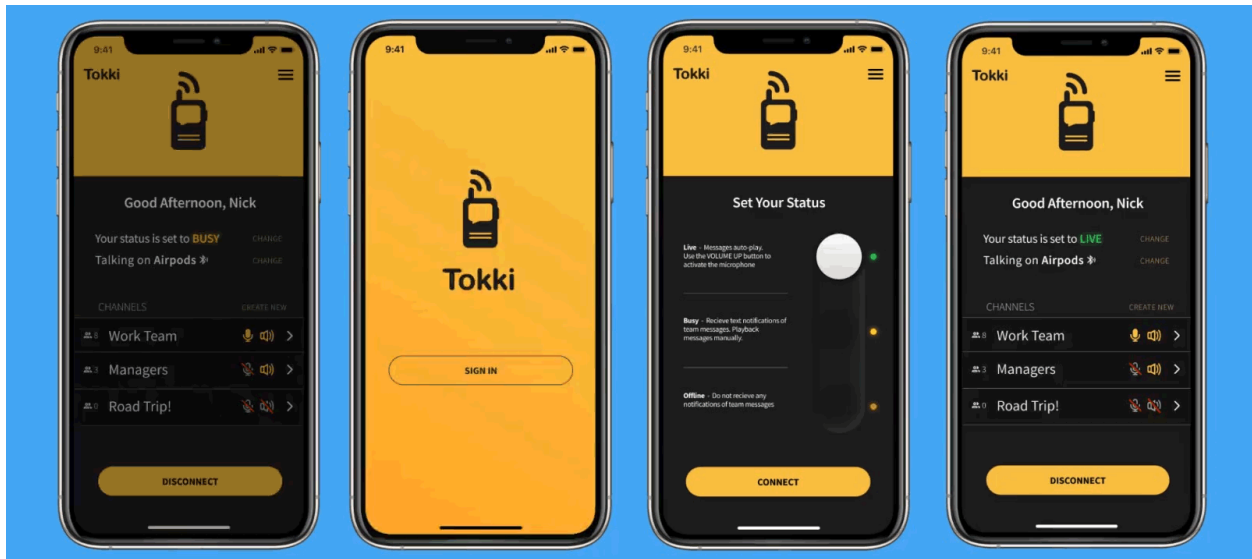
ICONIX khuyến khích viết use case ngắn gọn (2–3 đoạn), tập trung vào hành vi, tránh biến nó thành tài liệu yêu cầu khổng lồ. Mỗi use case là một câu chuyện rõ ràng, có đầu – giữa – cuối.

1.3.4. GUI Storyboarding — đưa người dùng “vào khung hình”

Một trong những điểm thực tế nhất của ICONIX là khuyến khích sử dụng GUI storyboard — các bản phác thảo giao diện mô phỏng trình tự tương tác. Con người thường khó hình dung một hệ thống chỉ qua lời nói. Nhưng nếu các bạn cho khách hàng và nhóm xem chuỗi các màn hình giao diện (dù chỉ là phác thảo), thì lập tức cuộc trò chuyện trở nên cụ thể hơn.

Ví dụ, khi xây dựng use case “Đặt vé xe”, nếu chỉ mô tả bằng văn bản, khách hàng có thể bỏ sót các tình huống như “nút quay lại”, “trường hợp chọn sai ghế”. Nhưng nếu có storyboard gồm 5 màn hình nối tiếp nhau, khách hàng sẽ dễ phát hiện ra lỗi logic hoặc yêu cầu bổ sung. Storyboard có thể là:

- Bản vẽ tay đơn giản.
- Slide PowerPoint.
- HTML prototype đơn giản không có backend.
- Hình chụp mockup từ Figma.



Điểm mấu chốt là: GUI storyboard giúp đảm bảo use case bám sát thực tế sử dụng và tránh bỏ sót luồng phụ.

1.3.5. Tổ chức & rà soát yêu cầu — Milestone 1

Khi domain model, use case và storyboard đã hoàn thành ở mức đầu tiên, ICONIX khuyến nghị thực hiện Requirements Review — cột mốc kiểm duyệt đầu tiên.

Mục tiêu của buổi này là:

- Đảm bảo domain model phản ánh được $\geq 80\%$ khái niệm quan trọng trong bài toán.
- Đảm bảo use case mô tả đủ luồng chính và luồng phụ.
- Đảm bảo storyboard và use case khớp nhau.
- Mời cả người dùng, stakeholder và kỹ thuật tham gia để rà soát đa chiều.

Đây không phải là một buổi họp hành mang tính hình thức. Nếu làm tốt, buổi review này sẽ giúp nhóm phát hiện rất nhiều hiểu lầm hoặc thiếu sót trước khi bước vào thiết kế — nơi chi phí sửa lỗi sẽ tăng gấp nhiều lần.

1.3.6. Sai lầm phổ biến của sinh viên & nhóm dự án

Ở Việt Nam, khi giảng về UML và yêu cầu phần mềm, tôi thường thấy sinh viên (và cả các nhóm startup) mắc 3 lỗi lớn:

1. Nhảy vào vẽ database thay vì domain model.
Họ nghĩ rằng vẽ bảng là đủ, nhưng database là thiết kế vật lý, không phản ánh hành vi hay ngữ nghĩa.

2. Viết use case như mô tả kỹ thuật.

Họ biến use case thành bản đặc tả dài ngoằng, khiến mất đi tính dễ hiểu và khả năng rà soát với người không kỹ thuật.

3. Không storyboard GUI.

Họ nghĩ giao diện là phần làm sau cùng, trong khi thực ra đây là công cụ mạnh để làm rõ yêu cầu sớm.

ICONIX Process đưa ra một chuỗi bước hợp lý để tránh những lỗi này:

→ Domain Model → Use Case → GUI Storyboard → Review.

Nhờ vậy, nhóm có một nền tảng vững chắc để bước vào giai đoạn phân tích và thiết kế.

Chốt lại, giai đoạn yêu cầu & mô hình hóa trong ICONIX giúp các bạn: (1) Xây dựng ngôn ngữ chung thông qua domain model. (2) Nắm rõ hành vi hệ thống thông qua use case. (3) Làm rõ giao diện và luồng tương tác bằng storyboard. (4) Đảm bảo mọi người hiểu đúng thông qua milestone review. Đây chính là **“trái tim” của ICONIX Process**. Mọi bước phía sau — từ robustness diagram đến sequence diagram — đều dựa trên những gì được xây dựng ở giai đoạn này.

1.4. Lược giản phân tích sơ bộ & Thiết kế

Khi các bạn đã có một tập use case rõ ràng, domain model đã hình thành, GUI storyboard đã vẽ, và toàn bộ yêu cầu đã qua một vòng kiểm duyệt nghiêm túc, thì đó chính là lúc nhóm bước vào **giai đoạn cầu nối**: từ yêu cầu → thiết kế.

Đây chính là khu vực “vùng xám” mà ICONIX Process xử lý rất tốt, bằng hai công cụ quan trọng: **Robustness Analysis** và **Sequence Diagramming**.

1.4.1. Robustness Analysis — cầu nối từ use case đến thiết kế

Trong thực tế phát triển phần mềm, có một khoảng cách rất lớn giữa “người dùng làm gì” (mô tả trong use case) và “hệ thống được cấu trúc ra sao” (mô tả trong class diagram). Nếu nhóm nhảy thẳng từ use case sang code, thì sẽ xảy ra hai tình huống phổ biến:

- Tình huống 1: Mỗi lập trình viên tự “dịch” use case theo cách hiểu riêng, dẫn đến thiết kế bị phân mảnh, mỗi phần của hệ thống giống như một hòn đảo độc lập.
- Tình huống 2: Nhóm dành nhiều thời gian để tranh luận xem nên chia lớp thể nào, mà không có công cụ trung gian để cấu trúc cuộc thảo luận → dễ dẫn tới thiết kế tùy hứng.

Robustness Analysis chính là công cụ trung gian đó.

Ý tưởng chính: Robustness Analysis được thực hiện bằng cách vẽ sơ đồ robustness diagram cho từng use case. Trên sơ đồ này, ta đặt ba loại đối tượng cơ bản:

1. **Boundary objects** – đối tượng biên giữa người dùng và hệ thống (ví dụ: màn hình, form nhập liệu, API endpoint).
2. **Entity objects** – các đối tượng thực thể (ví dụ: Vé, Chuyến Xe, Khách Hàng...).
3. **Controller objects** – các chức năng logic điều phối, xử lý luồng giữa boundary và entity (ví dụ: DatVeController).

Sơ đồ robustness thể hiện trình tự tương tác giữa các loại đối tượng này khi thực hiện use case. Các bạn có thể hình dung nó như một **bản phác thảo sơ khai của thiết kế hướng đối tượng**, được “kéo” trực tiếp ra từ use case.

Ví dụ: Với use case “Đặt vé xe”, sơ đồ robustness có thể gồm:

- **Boundary:** Màn hình chọn tuyến → màn hình chọn ghế → màn hình thanh toán.
- **Controller:** DatVeController (nhận yêu cầu từ GUI, kiểm tra ghế, tạo vé, xử lý thanh toán).
- **Entity:** Vé, Chuyến Xe, Khách Hàng.

Các mũi tên thể hiện sự tương tác: GUI gọi controller, controller truy xuất entity, cập nhật trạng thái, trả kết quả về GUI.

Tại sao bước này quan trọng?

Robustness Analysis giúp nhóm phát hiện sớm rất nhiều vấn đề:

- Thiếu lớp miền (ví dụ quên mô hình hóa Vé).
- Nhầm lẫn ranh giới – controller – entity.
- Logic bị mơ hồ hoặc vòng lặp luồng xử lý.
- Use case chưa đủ chi tiết để thiết kế.

Nói cách khác, đây là bước “lọc tạp chất” trước khi đúc khuôn thiết kế. Nếu bỏ qua bước này, rất dễ xảy ra tình trạng “thiết kế hời hợt” → code chắp vá.

1.4.2. Preliminary Design Review (PDR)

Khi sơ đồ robustness và domain model đã được cập nhật, ICONIX khuyến nghị thực hiện **PDR** – Preliminary Design Review. Đây là một checkpoint quan trọng nhằm đảm bảo:

- Mọi use case đã có sơ đồ robustness.
- Tất cả các entity và boundary trên sơ đồ đều tồn tại trong domain model.

- Dữ liệu luân chuyển hợp lý.
- Không có khoảng trống logic.

PDR đóng vai trò giống như việc “kiểm tra bản thiết kế nhà trước khi đổ móng”. Nếu qua được bước này, nhóm có thể tự tin bước vào thiết kế chi tiết.

1.4.3. Sequence Diagramming — từ hành vi đến cấu trúc

Sau khi các bạn đã có sơ đồ robustness, bước kế tiếp trong ICONIX là vẽ **sequence diagram** cho từng use case.

Ý tưởng chính: Sequence diagram thể hiện cách các đối tượng tương tác với nhau theo thời gian để thực hiện một chức năng cụ thể.

- Mỗi đối tượng là một “cột” (lifeline).
- Các thông điệp được biểu diễn bằng mũi tên.
- Trình tự từ trên xuống dưới thể hiện thứ tự thời gian.

Trong ICONIX, sequence diagram có hai nhiệm vụ chính:

1. **Phân bổ hành vi cho các lớp** (behavior allocation): quyết định mỗi chức năng sẽ được cài đặt trong lớp nào.
2. **Làm rõ dòng chảy logic** của use case ở mức thiết kế chi tiết, bao gồm cả luồng chính và luồng ngoại lệ.

Khi vẽ sequence diagram, các bạn sẽ thấy rõ:

- Lớp nào cần có phương thức nào.
- Các thông điệp qua lại yêu cầu tham số gì, trả về gì.
- Các lớp nào cần tương tác trực tiếp với nhau.

Từ đó, nhóm cập nhật **class diagram** để phản ánh đầy đủ cấu trúc và hành vi. Đây chính là lúc domain model “tiến hóa” thành thiết kế thực sự.

ICONIX còn có **CDR – Critical Design Review** sau bước này để đảm bảo mọi thứ khớp logic, nhưng trong phạm vi buổi học này, các bạn chỉ cần hiểu tư tưởng:

Chốt lại: Sequence diagram là cầu nối cuối cùng từ mô hình hành vi sang mô hình cấu trúc, trước khi lập trình bắt đầu.

Kết luận

Sau bài này, các bạn đã đi qua hành trình từ bối cảnh ra đời của ICONIX, đến bức tranh toàn cảnh của quy trình, và cuối cùng là giai đoạn yêu cầu – mô hình hóa – phân tích sơ bộ. Tư tưởng cốt lõi của ICONIX có thể tóm gọn trong một câu:

“Đưa nhóm phát triển đi qua khoảng trống từ use case đến code một cách có hệ thống, thực dụng và lặp lại được.”

Các bạn cần hiểu rằng ICONIX không chỉ là một bộ bước cụ thể, mà là một khung tư duy để đối phó với sự phức tạp của phát triển phần mềm: Đặt trọng tâm vào việc hiểu yêu cầu đúng ngay từ đầu. Dùng domain model và use case để tạo ngôn ngữ chung. Dùng storyboard để gắn chặt với hành vi thực tế của người dùng. Dùng robustness diagram để chuyển yêu cầu thành cấu trúc ban đầu. Dùng sequence diagram để hoàn thiện thiết kế hướng đối tượng. Dùng milestone review để kiểm soát chất lượng liên tục.

Câu hỏi ôn tập

1. Trình bày bối cảnh ra đời của ICONIX Process. ICONIX ra đời để giải quyết vấn đề gì trong phát triển phần mềm?
2. Giải thích vì sao UML đầy đủ (theo OMG) thường khó áp dụng nguyên vẹn trong các dự án thực tế.
3. ICONIX Process được mô tả bằng mấy giai đoạn chính? Hãy nêu tên và mục tiêu chính của từng giai đoạn.
4. Phân biệt functional requirements và behavioral requirements. Vì sao ICONIX nhấn mạnh việc tách biệt hai loại yêu cầu này?
5. Domain Model là gì? Vai trò của nó trong ICONIX Process là gì?
6. Tại sao GUI Storyboard lại quan trọng trong giai đoạn xác định yêu cầu?
7. Mốc kiểm duyệt (milestone) nào diễn ra sau khi hoàn thành domain model, use case và storyboard? Mục đích của nó là gì?
8. Mô tả ngắn gọn ý tưởng chính của Robustness Analysis và vì sao đây là bước “cầu nối” quan trọng.
9. So sánh vai trò của Robustness Diagram và Sequence Diagram trong ICONIX Process.
10. Vì sao ICONIX Process đặc biệt phù hợp với phương pháp Agile?